

EthioMorph: Technical Implementation of a Rule-Based Ge'ez Morphological Engine

Esubalew Chekol
College of Technology and Built Environment
Addis Ababa University
NLP Course Project

Abstract—Ge'ez verb conjugation follows strict patterns. Given a root like ቀተለ, every form (perfective, imperfective, jussive, imperative) can be computed by rearranging vowels and attaching affixes—no statistical model, but a typed root lexicon (lexicon.json) resolves verb class and a handful of edge cases that pure vowel heuristics cannot.

This paper documents EthioMorph, a Python system that does exactly that. It handles all eight verb types that traditional Ge'ez grammar recognizes: ቀተለ, ቀደሰ, ባረከ, ጦመረ, ሰሰየ, ከህለ, ማሕረከ, and ተንበለ. The core trick: Ethiopic Unicode assigns consecutive codepoints to vowel forms, so changing a vowel is just arithmetic. The rest is template matching, lexicon lookup, and special rules for laryngeal consonants and inherent root-initial ተ/ከ sequences.

Index Terms—Ge'ez, morphological analysis, NLP, Unicode processing, Semitic morphology, rule-based systems

I. Introduction

EthioMorph is a Ge'ez conjugator that works from first principles. Not a lookup table with thousands of pre-stored forms, and not a neural network trained on text. Conjugation is rule-driven via JSON templates; a compact lexicon maps each attested root to its verb class (type_a, type_d, type_tanbala, etc.).

Why is this even possible? Because Ge'ez is regular. The same root (ቀተለ, meaning “kill”) always conjugates the same way. There are eight verb types and maybe a dozen special cases for guttural consonants. Once you encode those rules, you can generate or analyze any verb.

A. What the System Does

Ge'ez verbs have two parts: a consonant root (usually 3 letters) and a vowel pattern. The root carries meaning, the pattern carries grammar.

Take q-t-l (ቀተለ, “kill”). Same three consonants, different vowels:

q-t-l + [1-1-1] → ቀተለ (past: “he killed”)
q-t-l + [yi-1-6-6] → ይቅጥሉ (present: “he kills”)

EthioMorph does this in both directions:

Analysis: ይቅጥሉ → root: ቀተለ, tense: imperfective, subject: 3rd plural masculine

Generation: (ቀተለ, imperfective, 3PM) → ይቅጥሉ

II. Unicode Mathematics for Ethiopic

A. The Ethiopic Block Structure

The Ethiopic Unicode block (U+1200–U+137F) organizes characters in a systematic pattern. Each consonant has 7 forms representing vowel variations:

TABLE I
Unicode Pattern for Consonant ቀ (qāf)

Order	Unicode	Character	Vowel
1	U+1240	ቀ	ā (schwa)
2	U+1241	ቁ	u
3	U+1242	ቂ	i
4	U+1243	ቃ	a
5	U+1244	ቄ	e
6	U+1245	ቅ	(none)
7	U+1246	ቆ	o

B. Vowel Arithmetic

Since vowel forms are consecutive in Unicode, you can do math on them:

$$\text{order}(c) = ((c - \text{base_offset}) \bmod 8) + 1 \quad (1)$$

$$\text{base}(c) = c - (\text{order}(c) - 1) \quad (2)$$

To change a vowel:

$$\text{revowelize}(\text{base}, \text{order}) = \text{base} + (\text{order} - 1) \quad (3)$$

Implementation:

```
1 def get_vowel_order(char):
2     """Extract vowel order (1-7) from
3     character."""
4     return ORDER_MAP.get(char, 1)
5
6 def devowelize(char):
7     """Return base consonant (1st order
8     form)."""
9     return DEVOWELIZATION_MAP.get(char,
10     char)
11
12 def get_char_by_order(base, order):
13     """Apply vowel order to base consonant.
14     """
15     return REVOWELIZATION_MAP.get((base,
16     order), base)
```

Example transformation:

```
devowelize(ቃ) = ቀ
get_vowel_order(ቃ) = 4
get_char_by_order(ቀ, 6) = ቀ
```

C. Ambiguous Vowel Glyphs

Some Ethiopic characters appear in more than one consonant row. The glyph **ጸ** is shared by both the **ጸ**-row and **ፀ**-row. Flat lookup maps can store only one base per character; EthioMorph pins **ጸ** to the **ጸ**-row after registration (AMBIGUOUS_VOWEL_DEFAULTS), matching the normalizer's **ፀ** → **ጸ** homophone map. Both valid bases are recorded in AMBIGUOUS_VOWELS.

III. The C1-C2-C3 Radical System

A. Naming the Radicals

C1, C2, C3 refer to the consonants:

- C1: First radical (consonant 1)
- C2: Second radical
- C3: Third radical
- C4: Fourth radical (quadriliterals only)

For ቀተሉ: C1=ቀ, C2=ተ, C3=ሉ

B. How Templates Work

Each conjugation form is just a vowel pattern plus optional prefix/suffix:

```
1 # Perfective 3rd singular masculine for
  Type A
2 template = {
3   "vowel_map": {"C1": 1, "C2": 1, "C3":
4     1},
5   "prefix": "",
6   "suffix": ""
7 }
8 # Result: C1(1st) + C2(1st) + C3(1st) = qā-
  tā-lā
```

```
1 # Imperfective 3rd singular masculine
2 template = {
3   "vowel_map": {"C1": 1, "C2": 6, "C3":
4     6},
5   "prefix": "yi",
6   "suffix": ""
7 }
8 # Result: yi + C1(1st) + C2(6th) + C3(6th)
  = yi-qā-t-l
```

C. Putting It Together

IV. Suffix Fusion Rules

A. When Suffixes Collide

When you attach a vowel suffix to a consonant-only ending (6th order), you can't just concatenate them:

$$C_{6th} + V_{suffix} \rightarrow C_{order(V)} \quad (4)$$

Example: ቀተሉ + ኡ ("they") ≠ ቀተሉኡ

Instead: ቀተሉ + ኡ → ቀተሉኡ

Algorithm 1 Generate a conjugated word

Require: root (string), tense, subject, verb_type (optional)

Ensure: conjugated word (string)

```
1: verb_type ← LEXICON[root] or detect_verb_home(root)
   or "type_a"
2: (surface_root, prefix_adj) ← handle_prefixes(root,
   verb_type) {Skip strip for type_tanbala and inherent
   ተ/ኡ roots}
3: template ← TEMPLATES[verb_type][tense][subject]
4: vowel_map ← template["vowel_map"]
5: radicals ← extract_radicals(surface_root)
6: result ← template["prefix"] + prefix_adj
7: for each position in ["C1", "C2", "C3", "C4"] do
8:   base ← devowelize(radicals[position])
9:   order ← vowel_map[position]
10:  if verb_type == "type_d" and tense == "perfective" and
   position == "C1" then
11:    if get_vowel_order(surface_root[C1]) == 6 then
12:      order ← 6 {Preserve bare C1, e.g. ከ in ከህለ}
13:    end if
14:  end if
15:  order ← apply_laryngeal_rules(base, order, position,
   tense, features)
16:  result ← result + get_char_by_order(base, order)
17: end for
18: result ← result + template["suffix"]
19: result ← apply_suffix_fusion(result)
20: return result
```

B. Fusion Algorithm

```
1 def apply_suffix_fusion(stem, suffix):
2   """Fuse 6th-order final vowel suffix.
   """
3   if not suffix:
4     return stem
5
6   last_char = stem[-1]
7   last_order = get_vowel_order(last_char)
8
9   # Check if stem ends in 6th order (
   consonant)
10  if last_order != 6:
11    return stem + suffix
12
13  # Map suffix to target order
14  suffix_vowel_map = {
15    '\u12A1': 2, # u-vowel suffix
16    '\u12A0': 4, # a-vowel suffix
17    '\u12A5': 5, # e-vowel suffix
18  }
19
20  first_suffix_char = suffix[0]
21  if first_suffix_char in
   suffix_vowel_map:
22    target_order = suffix_vowel_map[
   first_suffix_char]
23    base = devowelize(last_char)
24    fused = get_char_by_order(base,
   target_order)
```

```

25     return stem[:-1] + fused + suffix
      [1:]
26
27     return stem + suffix

```

V. Verb Home Detection Algorithm

A. The Eight Canonical Verb Types

In Classical Ge'ez grammar, verbs are classified into “homes” (ቤት) named after their canonical representative. EthioMorph implements all eight standard verb types:

TABLE II
The Eight Canonical Ge'ez Verb Types (Homes)

#	Type Head	Name	Description
1	ቀተለ	Type A	Strong triradical (C1 = 1st order)
2	ቀደሰ	Type B	Geminate (C2 doubles in conjugation)
3	ባረከ	Type C	Long vowel (C1 = 4th order ራብዕ)
4	ጦመረ	Type C-O	O-initial (C1 = 7th order ሳብዕ)
5	ሴሰየ	Weak-Final	Final radical is የ (weak)
6	ክህለ	Type D	Laryngeal/weak-pattern verbs (type_d)
7	ማሕረከ	Quadriliteral	Four consonant radicals (type_mahräka)
8	ተንበለ	Tänbälä	Quadriliteral; initial ተ is lexical, not derivational

B. How to Classify a Verb

Each verb type uses different conjugation templates. Classification combines algorithmic vowel-pattern detection with a lexicon override:

- How many radicals? (3 = triradical, 4 = quadriliteral)
- What vowel does C1 have? (5th order = Type B, 4th = Type C, 7th = Type C-O)
- For quadriliterals: is C2 at 6th order? (Tänbälä pattern, e.g. ተንበለ)
- Any laryngeals (ሀ, ሐ, ኀ, አ, ዐ) or weak consonants (ወ, የ)? (stored as *features*, not always the class)
- Lexicon entry when present (e.g. ክህለ → type_d, ረጸመ → type_d)

TABLE III
Algorithmic Classification Rules (before lexicon override)

Condition	Type	Example
radicals = 4, C2 order = 6	Tänbälä	ተንበለ
radicals = 4, else	Quadriliteral	ማሕረከ, ተጋደለ
C1 order = 7	Type C-O	ጦመረ
C1 order = 5	Type B	ቀደሰ
C1 order = 4	Type C	ባረከ
C3 ∈ {weak}	Weak-Final	ሴሰየ
Default (3 radicals)	Type A	ቀተለ, አመረ
Lexicon match	As listed	ክህለ → type_d

C. Detection Algorithm

The conjugator resolves the final class as LEXICON[root] or detect_verb_home(root).type or ``type\_a``. Roots such as ክህለ and ረጸመ are listed as type_d in the lexicon even when vowel heuristics alone would yield Type A.

Algorithm 2 Figure out which verb type this is (detect_verb_home)

Require: root (string with vowels)

Ensure: (type, features)

```

1: skeleton ← consonant_skeleton(root)
2: radicals ← len(skeleton)
3: c1_order ← get_vowel_order(root[0])
4: c2_order ← get_vowel_order(root[1]) {if present}
5: features ← detect_laryngeal_and_hollow(skeleton)
6: if radicals == 4 then
7:   if c2_order == 6 then
8:     return (“type_tanbala”, features) {ተንበለ}
9:   else
10:    return (“type_mahräka”, features) {ማሕረከ, ተጋደለ}
11:  end if
12: end if
13: if c1_order == 5 then
14:   return (“type_b”, features) {ቀደሰ}
15: else if c1_order == 4 then
16:   return (“type_c”, features) {ባረከ}
17: else if c1_order == 7 then
18:   return (“type_c_o”, features) {ጦመረ}
19: end if
20: return (“type_a”, features) {default; ክህለ uses type_d via lexicon}

```

D. Feature Detection

Some roots have special properties that affect conjugation:

```

1  LARYNGEALS = {'h', 'H', 'x', 'a', 'A'} #
   Gutturals
2  WEAK_CONSONANTS = {'w', 'y'}
3
4  def detect_features(root, radicals):
5      features = {}
6
7      # Check for laryngeal consonants
8      for rad in radicals:
9          base = devowelize(rad)
10         if base in LARYNGEAL_BASES:
11             features['has_laryngeal'] = True
12         break
13
14     # Check for hollow verb (weak C2)
15     if len(radicals) >= 2:
16         c2_base = devowelize(radicals[1])
17         if c2_base in WEAK_BASES:
18             features['is_hollow'] = True
19
20     # Check for weak initial
21     c1_base = devowelize(radicals[0])
22     if c1_base in WEAK_BASES:
23         features['weak_initial'] = True
24
25     return features

```

VI. Laryngeal Vowel Shift Rules

A. Gutturals Are Picky

Laryngeal consonants (**ሀ**, **ሐ**, **ኀ**, **አ**, **ዐ**) don't accept certain vowels. When you try to apply a 1st order vowel to a laryngeal, it shifts:

Laryngeal + 1st order → Laryngeal + 4th order (5)

B. Implementation

```
1 def apply_laryngeal_rules(char, position,
2   features):
3     """Shift vowels near laryngeal
4     consonants."""
5     if not features.get('has_laryngeal'):
6       return char
7
8     base = devowelize(char)
9     order = get_vowel_order(char)
10
11     # Laryngeals prefer 4th order in
12     # jussive
13     if is_laryngeal(base) and order == 1:
14       return get_char_by_order(base, 4)
15
16     return char
```

VII. Conjugator: Generation Edge Cases

A. Inherent vs. Derivational Prefixes

During *analysis*, the stemmer strips tense markers (**ይ**, **ተ**, **አ**, etc.). During *generation*, the conjugator must *not* strip leading **ተ/አ/ኀ** when they belong to the lexical root itself. Stripping is allowed only for derived stem templates (type_a_passive, *_causative, etc.).

```
1 # PREFIX_LOOKALIKES = (asn, aste, te, a, ne
2   ) in Ge'ez script
3
4 def should_strip_prefix(root, verb_type):
5     if verb_type == "type_tanbala":
6       return False # tenbele: te- is C1,
7       not passive
8     if verb_type and not
9       is_derived_stem_type(verb_type):
10       if root.startswith(
11         PREFIX_LOOKALIKES):
12         return False # e.g. amare,
13         tagedele
14     return True
```

Examples: (**ተንበላ**, perfective, 3sm) → **ተንበላ** (identity); (**ተንበላ**, jussive, 3sm) → **ይተንበላ** (not the erroneous double-marked **ይተንበላ** produced when **ተ-** was stripped).

B. Type D Perfective Templates

Type D verbs (**ከህላ** “be able”, **ፈጸመ** “finish”) use a distinct perfective vowel map: C1 defaults to 1st order, C2 to 6th (bare consonant), C3 to 1st.

```
1 "type_d": {
2   "perfective": {
3     "3sm": {"vowel_map": {"C1": 1, "C2": 6,
4       "C3": 1}},
5     "3sf": {"suffix": "t", "vowel_map": {"C1": 1, "C2": 6, "C3": 1}}
6   }
7 }
```

Guttural roots often store C1 at 6th order in the lexicon (**ከ** in **ከህላ**). The conjugator preserves that input order when C1 is already bare, so (**ከህላ**, perfective) → **ከህላ** while (**ፈጸመ**, perfective, 3sf) → **ፈጸመት** (C2 **ጸ** → **ዐ**).

C. Hollow Verb Surface Variants

Hollow verbs (hollow_w, hollow_y) admit *collapsed* imperative surfaces where the middle weak radical disappears and C1 carries a distinctive vowel order. For **አወረ** “to go,” the full imperative is **አወር**, but collapsed forms **አር** and the ultra-short **ሐ** also occur (C1 at 7th order signals the hidden **ወ**).

The conjugator emits these as non-destructive variants alongside the primary generated word:

```
1 {
2   "word": "<ho-wer>", # primary
3   "imperative (full surface)"
4   "variants": ["<hor>", "<ho>"], #
5   "collapsed alternates"
6   "derivation": { ... }
7 }
```

The simple expand API mirrors this as result["variants"][tense][person]. The UI (Morpher, Matrix, Compare, Root Tree) renders primary and collapsed forms together so users see both the template output and attested short surfaces.

Perfective hollow verbs collapse differently: (**አወረ**, perfective, 3sm) → **አረ** with C2 dropped and C1 at 1st order. The stemmer reconstructs such two-radical skeletons back to the lexicon root by inserting the middle weak consonant when the expanded form is listed as hollow_w or hollow_y.

VIII. Stemmer: Going Backwards

The Stemmer takes a conjugated word and figures out its root. It's the reverse of generation. Prefix stripping here applies to *tense and stem markers on inflected forms*, not to the inherent root-initial radicals guarded during generation.

A. The Steps

- 1) Normalize spelling (Ge'ez has some redundant characters)
- 2) Strip prefixes (**ይ**, **ተ**, **አ**, etc.)
- 3) Strip suffixes (**ኡ**, **ከ**, etc.)
- 4) Get the consonant skeleton
- 5) Restore any weak consonants that disappeared
- 6) Resolve to lexicon canonical orthography
- 7) Figure out tense/person from what you stripped
- 8) Classify the verb type

B. Canonical Root Resolution

Homophone normalization maps variant spellings to a single row for matching ($\text{ሐ} \rightarrow \text{ሀ}$, $\text{ሐ} \rightarrow \text{ሀ}$, etc.). That aids reconstruction, but the *reported* root should use the lexicon citation form when one exists. For example, ሐ , ሐC , and ሐወC all analyze to ሐወረ —not ሀወረ —because the lexicon lists the ሐ -row root and surface imperatives use ሐ (7th order on ሐ), not ሀ (7th order on ሀ).

```
1 def canonicalize_root(root):
2     if root in LEXICON:
3         return root
4     norm = normalize_geez(root)
5     if norm in LEXICON_NORMALIZED:
6         return LEXICON_NORMALIZED[norm]
7     if norm in HOLLOW_W_LOOKUP:
8         return HOLLOW_W_LOOKUP[norm]
9     if norm in HOLLOW_Y_LOOKUP:
10        return HOLLOW_Y_LOOKUP[norm]
11    return root
```

Two-radical perfective stems such as ሐረ and ቀመ are expanded by inserting ወ or የ when the resulting trilateral appears in the typed hollow-verb lists. Vowel-order signals (3rd, 5th, or 7th order on C1) take precedence over bare lexicon matches for ambiguous two-letter forms such as ጎሰ (collapsed ገየሰ) versus the unrelated root ጎሰ .

Skeletonization intentionally collapses vowel order and homophone rows for *matching* ($\text{ጣ/ጥ/ጠ} \rightarrow \text{ጠ}$; $\text{ኅ/ሀ} \rightarrow \text{ሀ}$). Citation recovery therefore uses a precomputed `skeleton_lookup`: normalized skeleton ሀጠአ maps back to lexicon entry ኅጥአ , so an inflected form such as ወአይትኅጣአ yields root ኅጥአ with meaning attached, not the bare skeleton string.

C. Checking Prefixes

You have to check longer prefixes first, or you'll make wrong matches:

```
1 PREFIXES = [
2     # Stem IV (Causative-Passive) - check
3     # first
4     'yaste', 'taste', 'naste', 'laste',
5     # Stem III (Causative)
6     'yas', 'tas', 'nas', 'las',
7     'ya', 'ta', 'na', 'a',
8     # Stem II (Passive)
9     'yit', 'tit', 'te',
10    # Stem I (Basic)
11    'yi', 'ti', 'ni', 'li'
12 ]
13 # Sorted by length descending
14 PREFIXES.sort(key=len, reverse=True)
```

D. Pattern Identification

```
1 def identify_verb_pattern(stem,
2     stripped_prefixes):
3     """Determine grammatical pattern from
4     affixes."""
5     # Check stem markers in priority order
6     if 'aste' in stripped_prefixes:
```

```
6         return {'stem': 4, 'name': '
7             Causative-Passive'}
8
9     if 'as' in stripped_prefixes or
10        has_causative_vowel(stem):
11        return {'stem': 3, 'name': '
12            Causative'}
13
14    if 'te' in stripped_prefixes or 'yit'
15        in stripped_prefixes:
16        return {'stem': 2, 'name': 'Passive
17            '}
18
19    return {'stem': 1, 'name': 'Basic'}
```

IX. Data Structures

A. Lexicon JSON

Each root entry supplies the verb class used by the conjugator (overriding `detect_verb_home` when present):

```
1 {
2     "roots": [
3         {"root": "<keHle>", "type": "type_d", "
4             meaning": "to be able"},
5         {"root": "<feTSeMe>", "type": "type_d",
6             "meaning": "to finish"},
7         {"root": "<tenbele>", "type": "
8             type_tanbala", "meaning": "to
9             intercede"}
10    ]
11 }
```

In the repository, root values are Ge'ez strings (ክህለ , ፈጽመ , ተገበለ , etc.).

B. Templates JSON

```
1 {
2     "type_a": {
3         "perfective": {
4             "3sm": {
5                 "vowel_map": {"C1": 1, "C2": 1, "C3
6                     ": 1},
7                 "prefix": "",
8                 "suffix": ""
9             },
10            "3sf": {
11                "vowel_map": {"C1": 1, "C2": 1, "C3
12                    ": 4},
13                "prefix": "",
14                "suffix": "t"
15            }
16        }
17        // ... 10 person forms
18    },
19    "imperfective": {
20        "3sm": {
21            "vowel_map": {"C1": 1, "C2": 6, "C3
22                ": 6},
23            "prefix": "yi",
24            "suffix": ""
25        }
26    }
27    // ...
28 }
```

```

26 "type_d": {
27   "perfective": {
28     "3sm": {"vowel_map": {"C1": 1, "C2":
29       6, "C3": 1}},
30     "3sf": {"vowel_map": {"C1": 1, "C2":
31       6, "C3": 1}, "suffix": "t"}
32   }
33 }

```

C. Lookup Maps

Three precomputed dictionaries make character lookups instant (with one documented ambiguity for **ʕ**):

```

1  # Character -> Base consonant
2  DEVOWELIZATION_MAP = {
3    '\u1240': '\u1240', # qe -> qe (
4      already base)
5    '\u1241': '\u1240', # qu -> qe
6    '\u1242': '\u1240', # qi -> qe
7    # ... all 300+ characters
8  }
9
10 # Character -> Vowel order (1-7)
11 ORDER_MAP = {
12   '\u1240': 1, # qe = 1st order
13   '\u1241': 2, # qu = 2nd order
14   # ...
15 }
16
17 # (Base, Order) -> Character
18 REVOWELIZATION_MAP = {
19   ('\u1240', 1): '\u1240',
20   ('\u1240', 2): '\u1241',
21   ('\u1240', 3): '\u1242',
22   # ...
23 }

```

The glyph **ʕ** (U+13BA) is registered in both the **ʕ**-row and **θ**-row; AMBIGUOUS_VOWELS records both bases, with **ʕ** pinned as the default in AMBIGUOUS_VOWEL_DEFAULTS.

X. System Architecture

XI. API Usage

A. Analysis Endpoint

```

1  POST /api/xray
2  {
3    "word": "yiqatlu"
4  }
5
6  Response:
7  {
8    "root": "qatala",
9    "pattern": "imperfective",
10   "stem": 1,
11   "person": "3pm",
12   "verb_type": "type_a"
13 }

```

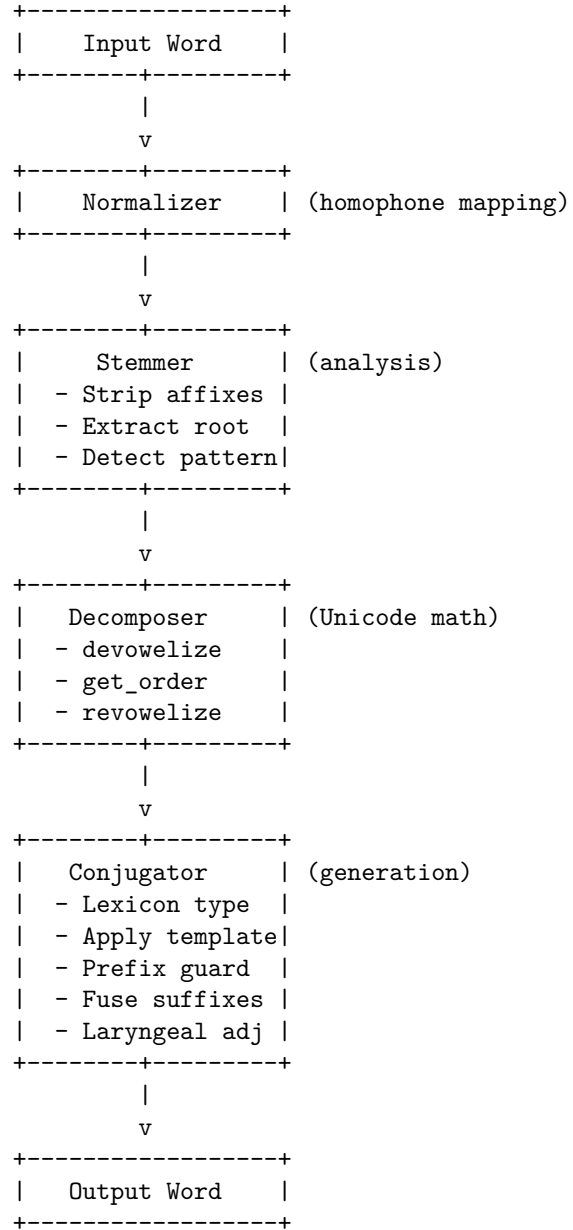


Fig. 1. How a word flows through the system

B. Generation Endpoint

```

1  POST /api/morph
2  {
3    "root": "qatala"
4  }
5
6  Response:
7  {
8    "perfective": {
9      "3sm": "qatala",
10     "3sf": "qatalat",
11     ...
12   },
13   "imperfective": {...},
14   "jussive": {...}

```

XII. Observations

Ge'ez morphology turns out to be more regular than expected. The Unicode trick (vowel forms are consecutive codepoints) turns what seems like a complex linguistic operation into basic arithmetic. Most of the code applies JSON templates, consults a typed lexicon, and handles edge cases: guttural vowel shifts, Type D perfective C2 muting, Tānbälā roots whose initial **ተ** must not be treated as a passive prefix, and ambiguous glyphs such as **ጸ**.

The system isn't complete. It handles the eight main verb types and basic stems, but derived forms and obscure patterns remain partial. Still, it conjugates most roots attested in the lexicon with rule-level accuracy on the regression suites for prefix look-alikes and Type D perfective forms.

Code: github.com/Esubaalew/EthioMorph